

ОДИН МЕТОД РОЗВ'ЯЗАННЯ ЗАДАЧІ РОЗДІЛЬНОСТІ МНОЖИН

О. М. Горошко, студент 4 курсу, групи МІ

Анотація. У роботі запропоновано узагальнений метод побудови перетину півплощин на площині та застосування цього методу до розв'язання задачі про повне розділення множин точок. В даному випадку застосування виявилось можливим завдяки зведенню задачі повного розділення до задачі перетину за допомогою системи лінійних нерівностей.

Abstract. In the paper we propose a generalized method for constructing the intersection of the half-plane on the plane and the application of this method to the problem of complete separation of sets of points. In this case, the application was possible by reducing the problem to the problem of complete separation of crossing through a system of linear inequalities.

1 Вступ

Постановка проблеми. В роботі розглядається один із підходів розв'язання задачі роздільності множин в евклідовому просторі в узагальненій постановці. Розв'язки задачі роздільності множин мають велике теоретичне і практичне значення, особливо, що стосується застосування, як в інформатиці так і прикладних науках. Наприклад, одним із таких застосувань є задачі розпізнавання образів з використанням нейронних мереж та класифікації об'єктів. Для ефективного навчання нейронної мережі необхідно, щоб інформація на вході перцептронів мала таку властивість, як лінійна роздільність множини її вхідних даних.

Важливим застосуванням, також, є побудова ефективних алгоритмів для задач геометричного та графічного моделювання складних явищ і процесів. Зокрема, це стосується моделювання складних теплофізичних процесів при зварюванні металевих конструкцій. Для розв'язання таких задач розробляються узагальнені ефективні паралельно-рекурсивні алгоритми [1-7]. Одним із підходів розробки таких алгоритмів є підхід, в основі якого лежить стратегія “розділяй та володарюй”. Та проблемою застосування цієї стратегії є лінійна нероздільність множини точок. Тому пошук ефективних підходів, які б дозволяли просто і ефективно розв'язувати, як класичну задачу роздільності множин так і узагальнену, яка полягає у визначенні всієї сукупності розділяючих прямих (гіперплощин) на заданій множині точок чи інших геометричних об'єктів, є актуальним.

Аналіз останніх досліджень. На сьогоднішній день для розв'язання задачі використовують два основних підходи, які ґрунтуються на структурах даних типу збалансованих дерев. Перший підхід, відомий як підхід без урахування часу [1, 2], представляє час як додатковий вимір простору, зберігає й працює з траєкторіями рухомих точок. Перевага даного підходу в тому, що структури даних, які він використовує, змінюються лише при зміні траєкторії точки або при додаванні чи видаленні точки. Недолік даного підходу очевидний: через використання простору більшої розмірності зростає час запиту [2]. В основі другого підходу [1, 6] лежить використання кінетичних структур даних [4] та динамічних дерев для рухомих точок. Він підтримує структуру даних у реальному часі: коли точки рухаються, структура змінюється. Не дивлячись на те, що точки рухаються неперервно, структура даних оновлюється лише в певні дискретні моменти часу, коли відбуваються деякі події, наприклад, коли координати деяких двох точок співпадають. Такий підхід дає ефективний час пошуку, проте потребує перебудови структури, навіть якщо траєкторія жодної точки не змінилась. Ще одним недоліком даного підходу є те, що він дає можливість відповідати лише на запити щодо поточної конфігурації точок. Звісно, можна використовувати перебудови для отримання результатів запитів з майбутнього, але при цьому до запитного часу додається час перебудови структури. Зазначимо, відповідно до робіт [1, 4, 6], що якщо множина S складається з n точок в $\mathbb{R}^1$, то, використовуючи кінетичні B -дерева, можна отримати результати запиту за час $O(\log n + k)$, де k – кількість точок, що потрапляє в запитний регіон. Перебудова структури даних здійснюється для $O(n^2)$ подій, кожна перебудова потребує $O(\log n)$ часу.

При зміні розмірності зростає лише час перебудови. Наприклад, для $\mathbb{R}^2$ він складає $O(\log^2 n)$, але, як зазначено в роботі [6], використовуючи бінарний пошук, його можна зменшити до часу $O(\frac{\log^2 n}{\log \log n})$.

З практичної точки зору, використання B -дерев досить складне. Можна використовувати $k-d$ -дерева, не дивлячись на те, що їх використання залишає пошуковий час незмінним, але загальний час погіршується, так як $k-d$ -дерева потребують більше перебудов, ніж B -дерева [3, 5, 6].

Зазначимо, що в загальній постановці (без суттєвих обмежень на функції руху, пошукові регіони та розмірність простору) задача є малодослідженою. Всі відомі алгоритми дозволяють виконувати регіональний пошук лише при умові, що закони руху об'єктів лінійно залежать від часу [1]. В роботах [5, 6] розглядаються варіанти нелінійного руху, але отримані результати мають теоретичний характер, так як клас нелінійних функцій руху, що розглянуто, дуже вузький. В роботах [1, 4, 6] визначено обчислювальну складність розв'язків даної задачі в просторах розмірності більше трьох та вказано оцінки пам'яті, що дозволяють будувати ефективні пошукові алгоритми. На основі розв'язків даної геометричної задачі в роботах [4–10] розглянуто можливе використання відповідних алгоритмів обчислювальної геометрії для побудови ефективних алгоритмів та структур даних для обробки швидкоплинної інформації в дискових системах управління базами даних.

Першим, хто взявся за розв'язання задачі ДЛТ в установлених обмеженнях, був Б. Чазеле. У 1983 році для розв'язання задачі він запропонував громіздку структуру даних, яку назвав *hive graph* [11]. Застосуванню традиційних статичних підходів, що задовольняли установленим **межам (методи трапецій, ланцюгів і Кіркпатріка)** заважало те, що вся логіка в них зосереджена в передобробці, що неприпустимо для динамічного випадку. У 1986 році Коул розробив метод, який зменшував об'єм використаної пам'яті на базі традиційного методу смуг Ліптона до $O(N)$ [12]. З'явилася ідея модифікувати алгоритм для динамічного випадку, і у 1986 році вийшла праця Сарнака і Тар'яна [13], в якій пропонувалось застосувати до методу Добкіна-Ліптона динамічну структуру, в якості якої було запропоновано червоно-чорне дерево. Саме втілення ідей Сарнака-Тар'яна з певними модифікаціями є *метою статті*.

Як альтернативу складному в реалізації червоно-чорному дереву у 2004 році запропоновано [14] метод перепозначення Дітца-Слетора [15]. Але цей метод має ваду: хоча в середньому він має хороші показники, в найгіршому випадку він не працює за логарифмічний час [16]. Тому в основі запропонованого методу лежить використання червоно-чорного дерева. Сучасні публікації пропонують в якості альтернативи методи, що засновуються на теорії вірогідності [17, 18]. Детально вивчається вузьке місце динамічної локалізації точки - проблема вводу-виводу [19, 20]. Крім того, стверджується, що при застосуванні суперкомп'ютерів можна подолати давнє обмеження на час запиту в $O(\log N)$ [21].

Новизна та ідея. В розглядуваній роботі запропоновано новий підхід, який узагальнює, як саму задачу роздільності так, і алгоритм її розв'язання.

Мета статті. Розробити узагальнений метод побудови повної множини розділяючих прямих для заданої множини точок в евклідовому просторі.

2 Основна частина.

Сформулюємо геометричну постановку задачі роздільності.

Постановка задачі повного розділення множин. Нехай задана множина S із N точок на площині (в E^d просторі). Необхідно визначити повну множину всіх розділяючих прямих (гіперплощин) l розбиття множини S на дві приблизно рівно потужні множини S_1 та S_2 .

Означення 1. Точка Z знаходиться вище заданої прямої k , якщо паралельна їй пряма, проведена через Z , перетинає вісь ординат вище прямої k . Аналогічно, точка Z лежить нижче прямої k , якщо паралельна пряма перетинає вісь OY нижче за k .

2.1. Побудова розв'язку задачі розділення.

Тоді, якщо l - розділяюча пряма, то всі точки однієї множини будуть розташовані вище неї, а іншої множини – нижче (окрім випадку, коли пряма паралельна осі ординат). Нехай $l: y = kx + b$ – деяка розділяюча пряма. Нехай всі точки множини S_1 розташовані над цією прямою, а множини S_2 – під прямою, відповідно. Нехай $Z_i(x_i, y_i)$ – довільна точка. Тоді $l_i: y = kx + b_i$ – пряма, що проходить через цю точку i паралельна до l . Ці прямі перетинають вісь OU відповідно в точках $(0, b_i)$ та $(0, b)$. Якщо $Z_i \in S_1$, то $b_i > b$, якщо $Z_i \in S_2$, то $b_i < b$. З рівняння прямої маємо, що $b_i = -kx_i + y_i$. Таким чином маємо систему нерівностей для двох випадків розташування розділяючої прямої:

$$b < -kx_i + y_i, \text{ для } Z_i \in S_1, \text{ і } b > -kx_i + y_i, \text{ для } Z_i \in S_2 \text{ (нижче } S_1) \quad (1)$$

$$b > -kx_i + y_i, \text{ для } Z_i \in S_1, \text{ і } b < -kx_i + y_i, \text{ для } Z_i \in S_2 \text{ (вище } S_2) \quad (2)$$

Множина розділяючих прямих виду $x = a$ визначається таким чином:

1) визначаються величини $X_{amin}, X_{amax}, X_{bmin}, X_{bmax}$ з умов:

$$X_{amin} = \min(x_i), x_i \in S_1, X_{amax} = \max(x_i), x_i \in S_1,$$

$$X_{bmin} = \min(x_i), x_i \in S_2, X_{bmax} = \max(x_i), x_i \in S_2;$$

2) якщо $X_{amin} > X_{bmax}$, то $a \in (X_{bmax}, X_{amin})$;

3) якщо $X_{bmin} > X_{amax}$, то $a \in (X_{amax}, X_{bmin})$;

4) інакше ця множина порожня.

Розв'язавши системи (1) і (2) та об'єднавши їх розв'язки з множиною вертикальних розділяючих прямих $x = a$, можна знайти множину прямих, які розділяють множини S_1 і S_2 . Геометрично така система нерівностей являє собою перетин півплощин, утворених прямими виду $b = -kx_i + y_i$. Таким чином, задача роздільності зводиться до задачі знаходження перетину півплощин.

Постановка задачі перетину. На площині задано множину півплощин A . Необхідно визначити опуклу область G , яка є перетином всіх півплощин.

Півплощина однозначно визначається прямою (далі - базова пряма), що є границею півплощини, та точкою на цій півплощині, яка не належить базовій прямій. Розділимо множину A на дві множини AU та AX . Множина AU і AX – це сукупність півплощин множини A , базові прямі яких описуються рівняннями вигляду $y = kx + b$ та $x = a$, відповідно. Множину AU розділимо на дві підмножини AU_u та AU_d за наступною ознакою: якщо півплощина з базовою прямою $y = k_i x + b_i$ містить точку $(0; b_{i+1})$ (точка, що лежить вище точки перетину базової прямої з віссю ординат), то вона належить множині AU_u , в іншому випадку - множині AU_d .

2.2 Побудова розв'язку задачі перетину.

Вилучимо паралельні прямі з множин AX , AU_u та AU_d . Півплощина вилучається, якщо вона повністю містить іншу півплощину. Таким чином, будь-які дві базові прямі півплощин, що містяться в одній множині перетинатимуться.

Попередня обробка та структура даних. Структура даних являє собою два двозв'язні списки, які містять коефіцієнти базових прямих: перший список містить всі прямі з множини AU_u , другий – AU_d . Списки впорядковуються за спаданням кутового коефіцієнта k_i (кути переглядаються за годинниковою стрілкою). Отже, для попередньої обробки достатньо відсортувати півплощини за кутом до осі ординат.

Алгоритм.

Для кожного списку виберемо базову пряму, що має найменше значення кутового коефіцієнта k_i . Назвемо її *опорною прямою* для вибраної множини. Позначимо через l_i пряму, яка відповідає кутовому коефіцієнту k_i . Побудова перетину півплощин з множин AU_u та AU_d здійснюється за два проходи.

Перший прохід. Переглядаємо список для A_{ud} , починаючи з кінця списку (k_i – найбільше). Якщо базова пряма, що розглядається, перетинає опорну пряму правіше попередньо розглянутої, то відповідна півплощина вилучається зі списку, бо перетин цієї півплощини з опорною повністю містить перетин попередньої півплощини з опорною, рис. 1,а.

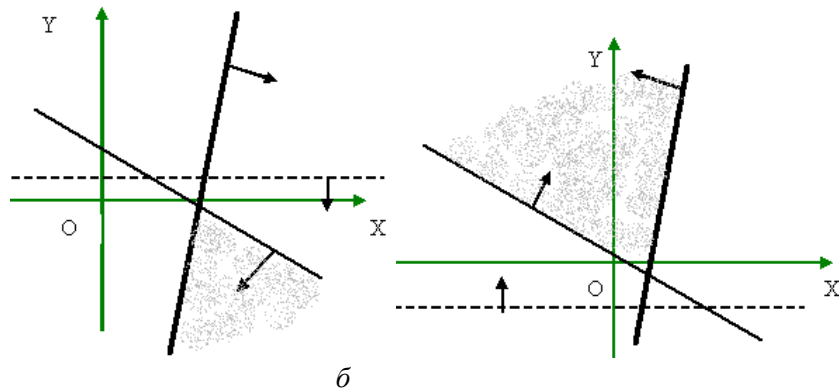


Рис. 1. Опорна півплощина – жирна лінія; півплощина, що вилучається – пунктирна лінія.

Аналогічний результат матимемо у випадку, коли розглядувана пряма перетинає опорну лівіше попередньої розглядуваної прямої, рис.1,б. Після першого проходу точка перетину будь-якої пари прямих, що належать одному списку, міститься у базовій півплощині.

Другий прохід. Прохід по списку для A_{ud} . Візьмемо три сусідні базові прямі з коефіцієнтами k_1, k_2, k_3 , відповідно. Нехай k_1 – це коефіцієнт з найменшим значенням – тобто прохід йде з початку списку, тоді:

- 1) якщо точка перетину прямих l_2 і l_3 лежить ближче до опорної прямої ніж точка перетину прямих l_1 і l_2 , то пряма l_2 вилучається;
- 2) якщо k_1 не перший елемент списку, то наступною трійкою прямих для перевірки обираються прямі з коефіцієнтами k_0, k_1, k_3 ;
- 3) інакше – трійка прямих з коефіцієнтами k_1, k_3, k_4 , рис.2, а.

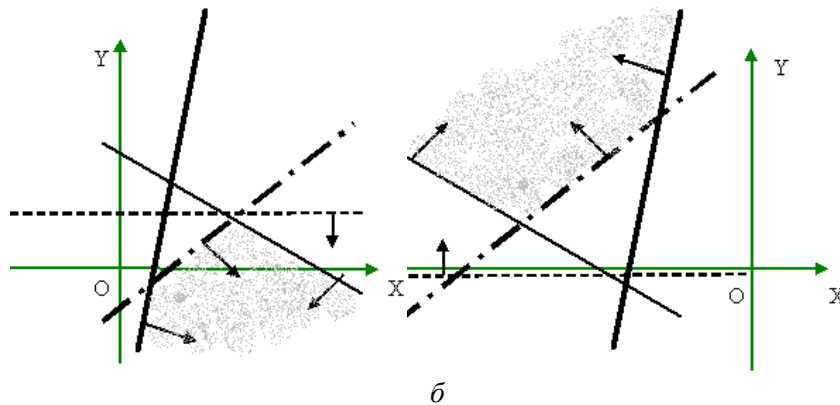


Рис. 2. Опорна пряма – жирна лінія, k_1 – пунктир з крапкою, k_2 – пунктир.

Прохід по списку для A_{ui} . У випадку проходу по списку A_{ui} матимемо аналогічний результат. Після другого проходу списки будуть містити півплощини, що утворюють опуклу область, і жодна півплощина не є зайвою, рис 2,б.

Заключний етап. Множина A_U визначається як перетин уже знайдених опуклих областей A_{ui} та A_{ud} за допомогою алгоритму, запропонованого в роботі [1]. Далі будується шукана область A перетином області A_U півплощинами з A_X , рис 3. Оскільки базові прямі всіх півплощин з A_X паралельні, то їх кількість не перевищує 2.

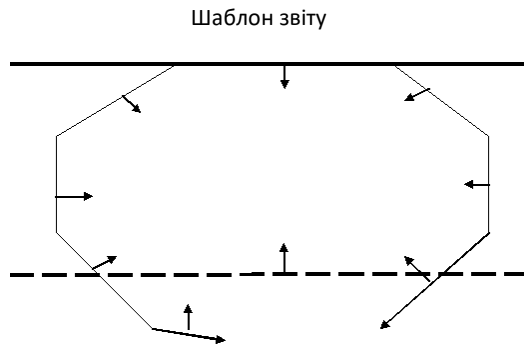


Рис.3. Базова пряма – жирна лінія, замикаюча пряма – пунктир.

3 Обґрунтування складності

Теорема 1. Часова складність розв'язання задачі про роздільність множини S на підмножини S_1 та S_2 становить $O(N)$ операцій за умови, що $O(N \log N)$ операцій піде на попередню обробку.

Доведення. Для зведення задачі розділення до задачі перетину потрібно на вхід алгоритму передати список півплощин, відповідні прямі яких мають коефіцієнти, що пропорційні до координат точок з множин. Таке перетворення здійснюється за лінійний час переглядом всіх точок з двох множин. Оскільки x -координата кожної точки є коефіцієнтом кута прямої, то сортування точок по x -координаті буде попередньою обробкою. На практиці, при розв'язанні задачі роздільності, точки, що потрапляють на вхід алгоритму, часто є відсортованими за x - чи y -координатою (наприклад, при роботі сканера), тому попередня обробка вхідних даних буде виконуватись лише за час $O(N)$.

4 Практична частина

Особливістю даної реалізації є *можливість роботи на незв'язному графі*. Програма здатна виконувати два типи локалізації: для замкненої і незамкненої областей. На рис.4 ілюструється інтерфейс програми для стандартного тесту. Алгоритмічна частина програмної реалізації написана на мові Java, для графічного відображення використовується пакет java.awt. Програма спирається на графічний інтерфейс і деякі функції обчислювальної геометрії, запропоновані Лудманом і Депуаном [9]. Для реалізації червоно-чорного дерева частково використовувався програмний код з М. Вайса [18].

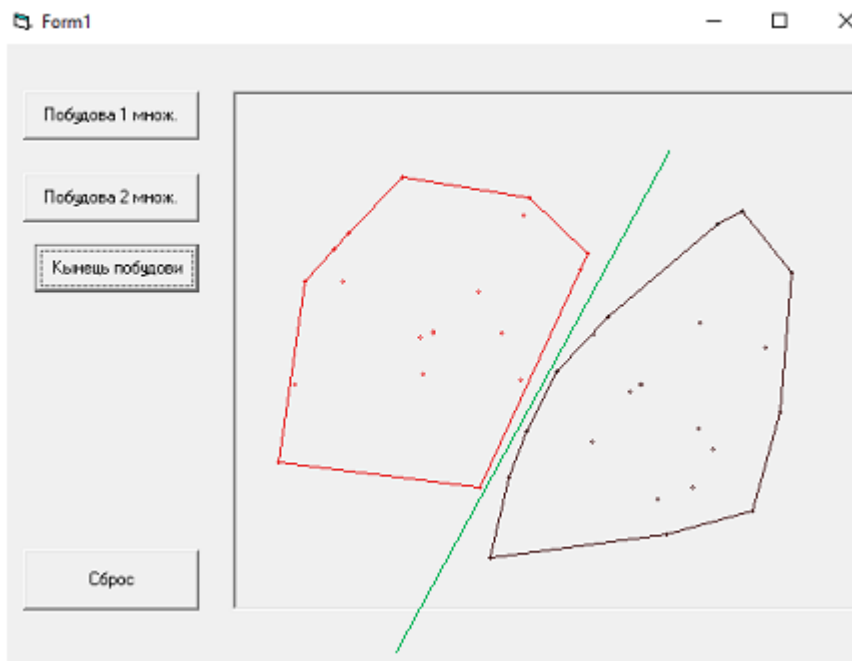


Рис.4.

5 Висновки

У роботі запропоновано узагальнений метод побудови повної множини розділяючих прямих для задачі роздільності множин точок на площині. Цей метод має місце і для випадку d -вимірного евклідового простору, з тією лише різницею, що будується множина розділяючих гіперплощин. Виявилось, що розроблений метод може бути ефективно застосований і до розв'язання інших задач обчислювальної геометрії: побудови опуклої оболонки множини точок, перетину опуклих многокутників, задачі лінійного програмування. Для визначення опуклої оболонки необхідно лише передати на вхід алгоритму список точок як множину S_1 , множина S_2 – порожня. Після другого проходу, точки які відповідають прямим, що входять до ланцюга, будуть утворювати верхню опуклу оболонку. Аналогічно, отримаємо нижню опуклу оболонку. Точки повинні бути відсортовані по x -координаті. Без урахування попередньої обробки, час знаходження опуклої оболонки – лінійний. Кожний многокутник являє собою перетин півплощин, які відсортовані за кутом. Зливши два списки та застосувавши алгоритм, отримаємо перетин. Час – лінійний, попередня обробка – лінійна (необхідно відновити півплощини, що утворюють відповідний многокутник).

Як відомо, розв'язок ЗЛП лежить на границі області допустимих значень. Знайшовши перетин обмежень, за лінійний час переглядаються всі вершини області допустимих значень. Зазначимо, що алгоритм дозволяє знайти не просто окремих розв'язок, а всю множину розв'язків. У випадку масових запитів, якщо області допустимих значень не змінюється, то побудувавши двійкове дерево для вершин її та знайшовши градієнт, можна знайти розв'язок за $O(\log N)$ часу на запит.

Список літератури

1. Герштейн В.М. Узагальнений підхід розв'язання деяких задач обчислювальної геометрії на основі рекурсивно-паралельної технології// Наукові Нотатки, Луцьк, 2008. Вип. 22, Ч. 2, С.344-349.
2. J.E. Goodman and J. O'Rourke, eds., Handbook of Discrete and Computational Geometry, Second Edition, Chapman and Hall/CRC Press, 2004.
3. B. Chazelle, N. Shouraboura. Bounds on the Size of Tetrahedralizations. Discrete & Computational Geometry, 1995, Vol. 14,(1), pp 429-444.
4. Franco P. Preparata. Computational Geometry: An Introduction. Springer-Verlag (1985, revised ed., 1991), 390 P.
5. Martin Kraus, Thomas Ertl Simplification of Nonconvex Tetrahedral Meshes, Mathematics and Visualization, Springer Berlin Heidelberg, LA, 2003 – pp 185-195
6. Ruppert, J.; Seidel, R., On the difficulty of tetrahedralizing 3-dimensional non-convex polyhedra, "On the difficulty of triangulating three-dimensional Nonconvex Polyhedra", Discrete Comput. Geom. 7: p.227–253. – 1992.
7. C. Hazlewood. Approximating constrained tetrahedralizations // Computer Aided Geometric Design, vol. 10, pp. 67-87, 1983.
8. 2. Шеймос М., Препарата Ф. Вычислительная геометрия: Введение. Пер. с англ. – М: Мир, 1989. – 478 с.
9. 3.. Megiddo N. Linear-time algorithms for linear programming in R^2 and related problems - SIAM Journal on Computing 12 (1983), No. 4, 759-776.
10. 4. M. de Berg, M. van Kreveld, M. Overmars, O. Schwarzkopf .Computational Geometry. Algorithms and Applications. Springer-Verlag, Berlin Heidelberg , Second Edition, 2000.-367 p.
11. Chazelle. B. Filtering search: A new approach to query-answering. In Proceedings of 24th Annual IEEE Symposium on Foundations of Computer Science (Tucson, Ariz., Nov. 7-9. 1983). 122-132.
12. Cole R. Searching and storing similar lists. - J. Algorithms, 1986.-P. 45-67.
13. Sarnak N., Tarjan R. E. Planar Point Location Using Persistent Search Trees. - Programming Techniques and Data Structures, 1986.-P. 345-367.
14. Tangwongsan K., Blleloch G., Acar U., Sridhar S, Vassilevska V.. Dynamic Point Location via Self-Adjusting Computation. - Aladdin Center, 2004
15. Dietz P. and Sleator D., "Two algorithms for maintaining order in a list,"// Proc. 19th Annual ACM Symp. on Theory of Computing (1987), 365-372.
16. Bender M. A., Cole R., Demaine E. D., Farach-Colton M., Zito J. Two simplified algorithms for maintaining order in a list// in: Proc. 10th ESA 2002. - pp. 78-97.
17. Arge L., Brodai G. S., Georgiadis L. Improved Dynamic Planar Point Location// In Proc. 47th Annual Symposium on Foundations of Computer Science, 2006.-pp. 23-46

18. Arya S., Malamatos T., Mount D.M. A Simple Entropy-Based Algorithm for Planar Point Location. - SIAM J. Comput., 2007.-P.34-45.
19. Agarwal P. K., Arge L., Brodal G. S., and Vitter J. S. "I/O-Efficient Dynamic Point Location in Monotone Planar Subdivisions"// in Proceedings of Tenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA'99.-1999,-pp.89-102.
20. de Berg M., Haverkort H., Thite S., Toma L. 1/O-EfRcient map overlay and point location in low-density subdivisions. // I n Tokuyama, editors, Proceedings of the 18th Annual International Symposium on Algorithms and Computation (ISAAC 2007), LNCS vol. 4835, pp. 500-511.
21. Chan T. M., Patrascu M. "Transdichotomous Results in Computational Geometry, I: Point Location in Sublogarithmic Time." - SIAM Journal on Computing 39.2 (2009) : 703-714.

Додатки